From Reactive to Predictive: A Federated Graph-Attention SAC Framework for Schedule-Driven WiFi Optimization in High-Density Transit Venues

Quang-Vinh Dang[a,1,*]

[a]*British University Vietnam, Hung Yen, Vietnam*

Abstract

Transit venues have predictable congestion: timetables announce when and where crowds will gather. We study whether this information can improve WiFi resource management and propose STAG-FedSAC, which combines a schedule-conditioned spatio-temporal graph attention forecaster (ST-GCAT), per-AP Soft Actor-Critic agents that choose power, channel and per-class airtime, and a two-level federation (zone averaging plus inter-zone distillation of policy outputs on a public reference set). We evaluate it in a simulator that, in contrast to the first version of this paper, models MAC contention (Bianchi), a channel-switching cost and a quantised airtime mapping onto 802.11 access categories; ignoring contention understates latency by about a factor of 1.8. All results are generated from logged runs. The timetable-conditioned forecaster reduces the load-forecast RMSE by 32% relative to persistence and by 27% relative to the same predictor without the schedule (surge windows: 31%); at the one-step horizon, simple forecasters are more

*Corresponding author

Email address: vinh.dq4@buv.edu.vn (Quang-Vinh Dang)

[1]ORCID: 0000-0002-3877-8024

accurate. The learned federated agents, which start from the operator's configuration, are stable and clearly better than learned DDPG baselines (reward 0.141 against 0.116 and 0.108), but they do not outperform the static configuration (0.181) in this simulator, and neither feeding the forecaster's embedding to the policy nor the joint predictor–policy gradient showed a benefit. Federation improves on isolated agents, and distillation-based inter-zone exchange needs 34× less wide-area traffic than weight-based hierarchical federation at a similar reward. Results are simulation-only, based on three training seeds, and the limitations are discussed in detail.
Keywords: WiFi resource management, Spatio-temporal graph attention, Soft Actor-Critic, Federated learning, Knowledge distillation, Transit venue networks, Deep reinforcement learning, Schedule-aware prediction

1. Introduction

Transit venues—airports, railway stations and bus terminals—are among the most demanding environments for WiFi. At peak hours a single departure lounge can host hundreds of users streaming video, making calls and uploading content over a handful of access points (APs) [1]. Unlike enterprise offices or university campuses, transit venues have a feature that a controller can exploit: part of the congestion is *announced in advance*. Departure and arrival timetables indicate when and where crowds will gather, with a temporal precision rarely available elsewhere. Commercial WLAN controllers, however, are largely *reactive*: they detect congestion after users experience degraded service and respond with load-balancing heuristics that ignore future schedule information [2].

We study whether schedule information can be turned into better radio resource decisions, and we organise the work around four gaps:

1. **Schedule unawareness**: load predictors treat AP load as a purely historical time series and discard timetable metadata that provides lead time before a surge [3].
2. **Reactive control**: rule-based controllers (strongest signal first, least loaded first) and DRL agents trained without look-ahead cannot pre-configure power and airtime before users arrive [4].
3. **Multi-class QoS**: VoIP needs low latency, video needs throughput and best-effort tolerates delay; this calls for a hybrid action (power, channel, per-class airtime) that plain continuous DDPG actions represent awkwardly [5].
4. **Distributed learning across administrative domains**: in a large venue, the zones can belong to different operators who do not pool per-user data, while isolated per-AP learners ignore what neighbouring APs have learned. Single-level federated averaging ignores zone structure [2].

We propose **STAG-FedSAC**, a framework that combines (i) a schedule-conditioned spatio-temporal graph attention Transformer (ST-GCAT) that forecasts AP load 30 minutes ahead, (ii) per-AP Soft Actor-Critic (SAC) agents whose state includes the ST-GCAT embedding, trained with an additional gradient path from the actors into the predictor, and (iii) a two-level federation (HierFed-KD) in which APs of a zone average their base models and zones exchange soft policy outputs on a public reference set instead of

weights. Figure 1 gives an overview and Section 3.1 defines the roles of the entities involved.

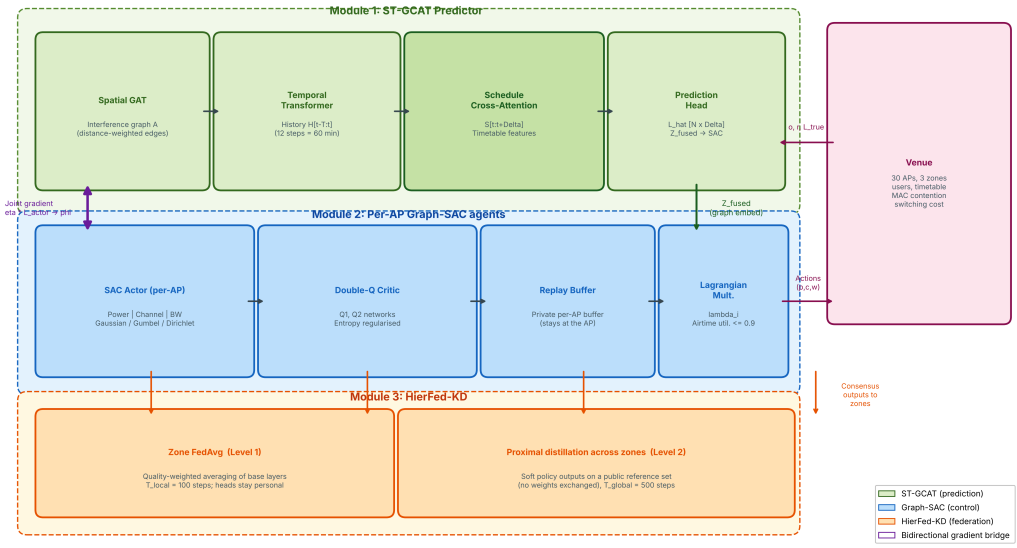


Figure 1: STAG-FedSAC overview. Module 1 (ST-GCAT) forecasts AP loads from the history of AP counters, the interference graph and the timetable, and produces embeddings $\mathbf{Z}_{\text{fused}}$. Module 2 (per-AP SAC agents) uses them in its state and chooses power, channel and per-class airtime; a gradient path ($\eta \cdot \mathcal{L}_{\text{actor}} \to \phi$) lets the control objective shape the predictor. Module 3 (HierFed-KD) federates the agents: model averaging inside a zone, distillation on a public reference set between zones. Per-user data and replay buffers stay at the AP.

Contributions. To the best of our knowledge, the combination below has not been reported for WiFi resource management; we evaluate each element separately with ablations and say where the evidence is weak.

- **Schedule-conditioned forecasting** (ST-GCAT): timetable features enter a graph-attention and Transformer predictor as cross-attention

keys and values, giving horizon-specific forecasts up to $\Delta = 6$ intervals ahead.

- **Graph-embedded SAC state**: the agent receives the full embedding $\mathbf{Z}_{\text{fused}} \in \mathbb{R}^{N \times \Delta \times d_h}$ instead of a scalar forecast.
- **Joint training** of predictor and policy: the predictor loss is augmented with a (scale-normalised) actor loss, $\mathcal{L}_{\text{pred}} = \text{MSE} + \eta\, \tilde{\mathcal{L}}_{\text{actor}}$.
- **HierFed-KD**: zone-level quality-weighted averaging plus inter-zone proximal distillation on a public reference set, with the communication volume measured byte by byte.
- **A more realistic evaluation** than a Shannon-capacity simulator: MAC contention (Bianchi model), channel-switching cost and an EDCA-style airtime mapping, with robustness tests for each (Section 4).

The experiments are run in a simulator; no measured traces are used. We therefore claim improvements *in simulation* only, and discuss in Section 5 what remains to be shown for a real deployment.

The paper is organised as follows. Section 2 reviews related work, Section 3 describes the framework, Section 4 the experiments, and Section 6 concludes.

2. Related Works

2.1. Spatio-Temporal Prediction for Wireless and Wi-Fi Networks

Accurate prediction of access-point (AP) load and wireless traffic is a prerequisite for proactive resource management, and spatio-temporal deep

learning has become the dominant approach. Shaabanzadeh and Sánchez-González [6] evaluate a broad family of architectures (CNN, GRU, LSTM and Transformer) for spatio-temporal Wi-Fi load prediction at the individual AP level on real campus measurements spanning 100 APs over three months, fusing spatial neighbour information only when inter-AP correlations are statistically significant.

At cellular scale, Cai et al. [7] propose DBSTGNN-Att, a dual-branch spatio-temporal graph neural network with self-attention that fuses location and point-of-interest features through an attributed k-nearest-neighbour graph, with superior short-, mid- and long-term predictions on the Milan Telecom Italia dataset. Lin et al. [8] unify normal, target-alarm and neighbour-alarm network states by transfer learning over spatio-temporal graph neural networks for multi-scenario cellular KPI prediction. Xiao et al. [9] combine a diffusion convolutional GRU with multi-head attention to capture spatial diffusion, temporal periodicity and external influence in cellular traffic.

The work closest in spirit to ours is STECA-GCN [10], which incorporates an event dimension via cross-attention in a spatio-temporal graph model and feeds the output into a DDPG-based load-balancing agent; its event features are, however, coarse binary workday/holiday labels. Among the works we reviewed, we did not find one that uses structured transit timetable data (arrival and departure times and passenger counts) as cross-attention context for Wi-Fi AP load prediction; this is the setting of our ST-GCAT predictor.

2.2. Deep Reinforcement Learning for Wireless Resource Allocation

Deep reinforcement learning has been widely applied to continuous resource-management problems in wireless networks. Yang et al. [11] identify a discounted-reward mismatch in prior wireless RL formulations and propose ARO-SAC, an average-reward Soft Actor-Critic variant for RAN slicing, with a 15% gain over conventional SAC on the NetworkGym simulator. For energy efficiency in RIS-assisted UAV systems, Adhikari et al. [12] jointly optimise phase shifts and 3D trajectory with twin delayed DDPG, benchmarking against SAC, DDPG and double DQN. Wang et al. [13] address 6G mmWave massive MIMO-NOMA with a hybrid DRL framework that pairs dueling DQN for discrete subchannel allocation with DDPG for continuous power allocation. Pi and Ren [14] develop HMSAC, a heterogeneous-action multi-agent SAC with an embedded attention mechanism that jointly optimises channels, power levels and bandwidths for NOMA uplink in vehicular networks. Qi et al. [15] formulate a multi-QoS DRL problem for RIS-aided IoV networks that minimises age of information under vehicle-to-vehicle reliability constraints. In the works we reviewed, the environment state is a scalar or flat vector (channel gains, per-slice metrics, positions); we did not find a work that builds the RL state from the dense, spatially structured embedding of a spatio-temporal predictor, which is what our Graph-SAC state does.

2.3. Federated Learning for Wireless Networks

Azimi-Abarghouyi and Fodor [16] introduce scalable hierarchical over-the-air federated learning with a two-level cluster/global aggregation scheme and convergence bounds for heterogeneous wireless topologies. Pervej et al. [17] address bandwidth scarcity and device heterogeneity in hierarchical FL by

adaptive model pruning during inter-level aggregation. At the aggregation-mechanism level, Shao et al. [18] propose Selective-FD, which selectively shares soft-label knowledge without a well-trained teacher, and Pang et al. [3] discuss how federated learning and knowledge distillation can be combined in wireless systems. Liang et al. [19] jointly optimise communication energy and training latency in over-the-air FL with a DRL algorithm, and Wu et al. [20] study age-based client selection and resource allocation for FL over NOMA networks. Li et al. [21] use knowledge distillation for asynchronous federated reinforcement learning in IoT. The hierarchical schemes above average or prune parameters, Selective-FD is formulated for classification, and the FL-wireless co-design works use flat FedAvg-style aggregation. We did not find a scheme that combines zone-level averaging, inter-zone distillation of *policy outputs* on a public reference set and Wi-Fi resource optimisation as the target application; we do not claim that none exists.

2.4. Graph Neural Networks in Networking

Dai et al. [22] survey graph-based resource management in wireless networks (GCN, GAT, GraphSAGE, MPNN) and identify GNN-enabled DRL as a major open direction. Liu et al. [23] integrate graph attention with multi-agent RL to address partial observability and non-stationarity. Ji et al. [24] combine GraphSAGE with DRL so that low-dimensional graph embeddings serve directly as the distributed agents' state for channel selection and power control in C-V2X. Marwani and Kaddoum [25] model multi-link interference as a graph and train a GNN to solve power control and spectrum allocation jointly, and Li et al. [26] use a heterogeneous GNN with distinct node and edge types for cell-free massive MIMO power allocation.

Ji et al. [24] is architecturally the closest to ours, since GNN embeddings are the RL state; it uses a static interference-topology convolution rather than a forecast-conditioned spatio-temporal embedding, employs DQN rather than SAC, and has no federated training. We did not find prior work that passes the output of a schedule-conditioned graph forecaster into the state of a SAC policy.

3. Methodology

This section first fixes the system model (who observes what, who decides, and what is shared), then defines the control problem, and finally describes the three modules of STAG-FedSAC. Every symbol is defined where it is first used; Table 1 collects them.

3.1. System Model and Roles

We consider a venue with N access points (APs) grouped into Z *zones* (e.g. departure hall, arrival hall, concourse). Three kinds of entities take part (Fig. 1):

- **Per-AP agent.** Each AP i is controlled by its own Soft Actor-Critic (SAC) agent: an actor π_i, two critics Q_i^1, Q_i^2 with target copies, and a private replay buffer $\mathcal{B}_i$. Agents do not share a policy by default; π_i keeps a *personal* head for the traffic of its own area. An agent runs on the AP itself or on the controller of the zone, observes only the quantities listed in Table 2, and every control interval chooses the action of *its* AP (Section 3.2).

- **Zone controller** (one per zone). Aggregates the models of the APs of its zone (Level 1 of HierFed-KD).
- **Venue controller** (one per venue). Hosts the ST-GCAT predictor, which needs AP-level load counters of all APs and the timetable; it also coordinates the zones (Level 2) and serves the public reference set used for distillation.

Time and data flow. Time advances in control intervals of 5 minutes. At the start of interval t, every AP reports its counters (Table 2) to the venue controller; ST-GCAT turns the last $T = 12$ intervals (one hour) of counters of all APs, the interference graph and the timetable of the next $\Delta = 6$ intervals into one embedding per AP; each agent then selects its action, and the reward is computed from the counters of the interval that follows.

What is federated, and why. Per-user data (association records, device identifiers, traffic mix, mobility traces) exist only to train the AP agents, and in a large venue the zones are frequently run by different parties (terminal operator, airline lounges, retail tenants) who cannot pool such data. Federated learning is therefore used to let agents benefit from each other's experience while *transitions, per-user records and replay buffers never leave the AP*; only (parts of) model parameters or, at Level 2, soft policy outputs on a public reference set are exchanged. The predictor consumes AP-level aggregates only (counts and averages, no user identifiers), which are already collected by any WLAN controller. We want to be explicit about the scope of this protection: it is *data minimisation*, not a formal privacy guarantee. We do not add differential privacy or secure aggregation, and exchanged parameters

or outputs can still leak information about local data. Section 4.7 quantifies what federation buys in learning performance and communication, by comparing it with isolated learners and with a (privacy-violating) pooled-experience learner.

Table 1: Main notation.

Symbol	**Meaning**
N, Z, C	number of APs, zones, channels
T, Δ	history length (12 intervals) and forecast horizon (6 intervals)
$\mathbf{H}$, $\mathbf{S}$, $\mathbf{A}$	AP history, schedule tensor, interference graph
L_i^t, $\hat{L}_i^t$	normalised load of AP i and its forecast
$\mathbf{Z}_{\text{fused}}$, d_h	ST-GCAT embedding and its width (128)
$\mathbf{s}_i$, $\mathbf{a}_i = (p_i, c_i, \mathbf{w}_i)$	state and action of AP i (power, channel, airtime budget)
u_i, λ_i	airtime utilisation and Lagrange multiplier of AP i
$\alpha, \beta, \gamma, \delta, \delta_p$	reward weights
η, κ	joint-gradient weight and embedding scale
μ, T_{local}, T_{global}	proximal weight and the two federation periods
M, $\mathcal{S}_{\text{ref}}$	size and set of public reference states
t_{sw}	channel-switch outage

3.2. Control Problem

Action. At each interval AP i chooses $\mathbf{a}_i^t = (p_i, c_i, \mathbf{w}_i)$: transmit power $p_i \in [P_{\min}, P_{\max}] = [5, 23]$ dBm, operating channel $c_i \in \{1, \ldots, C\}$ ($C = 11$ orthogonal 20 MHz channels), and an airtime budget $\mathbf{w}_i \in \Delta^3$ over the three QoS classes (VoIP, video, best-effort). How c_i and $\mathbf{w}_i$ are realised on commodity hardware, and what they cost, is described in Section 3.7.

Table 2: Information available to the framework at control interval t (per AP unless stated).

Input	**Content**
Local features $\mathbf{h}_i^t$	normalised user load L_i, throughput, mean latency, airtime utilisation u_i, numbe
History $\mathbf{H}$	$\mathbf{h}_i^{t-T+1}, \ldots, \mathbf{h}_i^t$ of all APs, $T = 12$ (oldest frame repeated at episode start)
Interference graph $\mathbf{A}$	weight $a_{ij} = 1 - d_{ij}/80\,\text{m}$ for $d_{ij} < 80$ m, else 0
QoS mix $\mathbf{q}_i^t$	fractions of VoIP / video / best-effort users
Radio context $\mathbf{r}_i^t$	own channel (one-hot), users and co-channel APs sensed in its collision domain (
Schedule $\mathbf{S}^{t:t+\Delta}$	8 features per AP and future interval derived from the timetable (Table 3)

Table 3: Schedule features $\mathbf{S}_i^\tau$ of AP i for a future interval τ (events of the zone of AP i; normalising constants in brackets). A timetable event is a departure or arrival with a time t_e and a number of passengers n_e.

#	Feature
1, 2	departure / arrival surge $\sum_e n_e \exp\big(-(t_e-\tau)^2/2\sigma^2\big)$, $\sigma = 2$ intervals [300]
3	1 if an event lies within ± 3 intervals of τ
4	intervals until the next departure (0 if none) [12]
5	intervals since the last arrival (0 if none) [12]
6	occupancy forecast $\min(1, (f_1 + f_2)/300)$
7	number of events of the zone in the next Δ intervals [3]
8	zone index / 2

Reward. The reward of AP i is

$$r_i^t = \alpha\, r_i^{\mathrm{tp}} + \beta\, r_i^{\mathrm{lat}} + \gamma\, r_i^{\mathrm{fair}} + \delta\, r_i^{\mathrm{qos}} + \delta_p\, b_i^{\mathrm{pred}}, \tag{1}$$

with every term on a unit scale:

$$r_i^{\mathrm{tp}} = \min\bigl(1,\, X_i / X_{\mathrm{ref}}\bigr), \qquad X_i = \text{aggregate throughput of AP } i \text{ [Mbps]},\ X_{\mathrm{r}}$$

$$r_i^{\mathrm{lat}} = -\min\bigl(1,\, \ell_i / \ell_{\mathrm{ref}}\bigr), \qquad \ell_i = \text{mean user latency [ms]},\ \ell_{\mathrm{ref}} = 100,$$

$$r_i^{\mathrm{fair}} = \frac{\bigl(\sum_u \varsigma_u\bigr)^2}{n_i \sum_u \varsigma_u^2}, \qquad \varsigma_u = \text{throughput of user } u \,/\, \text{its offered rate},$$

$$r_i^{\mathrm{qos}} = -\frac{\bigl|\{u :\ u \text{ violates its class requirement}\}\bigr|}{n_i}, \qquad \text{requirement: SINR} \geq \theta_c,\ \text{latency} \leq \ell_c^{\mathrm{SLA}},\ \varsigma_u$$

$$b_i^{\mathrm{pred}} = -\bigl|\hat{L}_i^{t+1} - L_i^{t+1}\bigr|, \qquad \text{(training signal only; not part of the evaluati}$$

The weights $\alpha = 0.40$, $\beta = 0.25$, $\gamma = 0.20$, $\delta = 0.05$, $\delta_p = 0.10$ sum to one and encode an operator priority order (throughput, then latency, then fairness; the QoS-violation term is small because violations already lower the throughput and latency terms, and the prediction term is a small auxiliary shaping signal). They are a design choice, not the output of a derivation: we do not claim they are optimal, and Section 4.9 reports how conclusions change under three other weightings. SAC is trained on the reward multiplied by a constant scale $R_{\mathrm{s}} = 10$ that matches the reward magnitude to the entropy term; *evaluation always uses the unscaled environment reward of Eq.* (1) *without* b^{pred}, so that methods without a predictor are scored identically.

Constraint. Let $\rho_i^{\mathrm{need}} = \sum_{u \in \mathcal{U}_i} d_u / R_u$ be the airtime AP i needs to carry the offered rates d_u at the PHY rates R_u of its users, and ρ_i^{avail} the airtime it actually obtains (its share of the collision domain times the MAC efficiency, Section 4.1). We require the *airtime utilisation* $u_i^t = \rho_i^{\mathrm{need}} / \rho_i^{\mathrm{avail}} \leq u_{\max} = 0.9$.

In contrast to the number of associated users, u_i *can be controlled* by power, channel and airtime split, which is what makes it a meaningful constraint. It is enforced by an adaptive multiplier,

$$\lambda_i^{t+1} = \mathrm{clip}\Big(\lambda_i^t + \rho_\lambda \, \min\big(u_i^t - u_{\max},\, 0.3\big),\, 0,\, \lambda_{\max}\Big), \qquad \rho_\lambda = 0.01,\ \lambda_{\max} = 1, \tag{2}$$

and the penalty $-\lambda_i \min\big(\max(u_i - u_{\max}, 0), 0.3\big)$ is added to the reward the agent learns from (it is not part of the evaluation reward). The bound on λ_i and the clipped violation keep the penalty on the scale of the reward when an AP is persistently overloaded and the constraint cannot be met.

3.3. Module 1: ST-GCAT Predictor

ST-GCAT forecasts the normalised load $L_i^{t+1:t+\Delta}$ of every AP ($L_i = \min(1, \mathrm{users}_i/60)$) and produces the embedding $\mathbf{Z}_{\mathrm{fused}}$ used as part of the agents' state. Table 4 gives the exact sizes of all learned maps.

Table 4: ST-GCAT layer dimensions ($d_h = 128$, $K = 4$ heads, $d_k = d_h/K = 32$, $F = 5$ input features, $D_s = 8$ schedule features, $\Delta = 6$).

Map	**Shape**	**Role**
$\mathbf{W}_{\mathrm{in}}$, $\mathbf{W}_s$	$F \times d_h$, $D_s \times d_h$	input projections of $\mathbf{H}$ and $\mathbf{S}$ (the
$\mathbf{W}^{\mathrm{sp}}$	$d_h \times d_h$	GAT linear map, output split into
$\mathbf{a}^k = [\mathbf{a}_l^k; \mathbf{a}_r^k]$, $k = 1..K$	$2d_k = 64$ per head	GAT attention vector of head k (c
Temporal Transformer	2 layers, K heads, feed-forward width $4d_h$	self-attention over the T time step
Cross-attention	K heads, $d_h = 128$	queries from the temporal summar
Prediction head	$d_h \to 64 \to 1$, ReLU, sigmoid	maps each $\mathbf{Z}_{i,k}$ to a load in $[0, 1]$

Spatial GAT. For every time step of the history, node features are mixed over the interference graph. For head k and nodes i, j (with $\mathbf{g}_i = \mathbf{W}^{\mathrm{sp}}\mathbf{x}_i$ and $\mathbf{g}_i^k \in \mathbb{R}^{d_k}$ its k-th block),

$$e_{ij}^k = \mathrm{LeakyReLU}\big(\mathbf{a}^{k\top}[\mathbf{g}_i^k \,\|\, \mathbf{g}_j^k]\big) + 0.1\, a_{ij} = \mathrm{LeakyReLU}\big(\mathbf{a}_l^{k\top}\mathbf{g}_i^k + \mathbf{a}_r^{k\top}\mathbf{g}_j^k\big) + 0.1\, a_{ij}, \quad \alpha_{ij}^k = \frac{\exp e_{ij}^k}{\sum_{m\in\mathcal{N}_i\cup\{i\}} e} \tag{3}$$

and $\mathbf{h}_i^{\mathrm{sp}} = \mathrm{ELU}\big(\|_k \sum_j \alpha_{ij}^k \mathbf{g}_j^k\big) \in \mathbb{R}^{d_h}$, where $\mathcal{N}_i = \{j : a_{ij} > 0\}$ and $\|$ is concatenation over the heads.

Temporal Transformer and schedule cross-attention. For each AP, a Transformer encoder runs over the T spatially mixed frames and its last output $\mathbf{u}_i \in \mathbb{R}^{d_h}$ summarises the history. The summary is repeated for the Δ forecast steps and then *queries* the schedule through multi-head cross-attention,

$$\mathbf{Z}_i = \mathrm{LN}\big(\mathbf{U}_i + \mathrm{CrossAttn}(Q = \mathbf{U}_i,\, K = \mathbf{S}_i,\, V = \mathbf{S}_i)\big) \text{ followed by a residual feed-forward block,} \tag{4}$$

with $\mathbf{U}_i \in \mathbb{R}^{\Delta\times d_h}$ and $\mathbf{S}_i$ projected to $\mathbb{R}^{\Delta\times d_h}$. The embedding $\mathbf{Z}_{\mathrm{fused}} = [\mathbf{Z}_1, \ldots, \mathbf{Z}_N] \in \mathbb{R}^{N\times\Delta\times d_h}$ is passed to the agents. Because the summary $\mathbf{U}_i$ is the same for all Δ steps, the horizon-specific information in $\mathbf{Z}_i$ comes from the schedule; a schedule-free variant (ablation A2, Section 4.6) therefore has no horizon-specific signal other than the positional structure learned by the head.

Training targets. The targets of the predictor are the *true* loads L_i^{t+1+k}, $k = 0..\Delta-1$. A sample $(\mathbf{H}, \mathbf{S}, \text{targets})$ is stored when the corresponding future intervals have been observed (at the end of the episode; steps beyond the end of the episode are masked), and the predictor is trained on minibatches of 16 stored samples.

3.4. Module 2: Graph-SAC Agents

State. The state of agent i concatenates its local features, its slice of the ST-GCAT embedding, its QoS mix, its next-interval schedule features and its radio context:

$$\mathbf{s}_i^t = \big[\mathbf{h}_i^t,\ \kappa\,\mathrm{vec}(\mathbf{Z}_{\mathrm{fused},i}^t),\ \mathbf{q}_i^t,\ \mathbf{S}_i^{t+1},\ \mathbf{r}_i^t\big] \in \mathbb{R}^{5+768+3+8+13=797}. \tag{5}$$

The scalar $\kappa = 0.03$ scales the embedding block. The layer-normalised embedding has 768 entries of unit scale (norm ≈ 28), whereas the other 29 inputs have a combined norm of about 3; unscaled, the embedding dominates the first layer of the actor and critic. We chose κ among $\{1, 0.125, 0.03\}$ on validation episodes of one seed (seeds disjoint from the test seeds); larger values gave lower validation reward. This choice is part of the method and its consequences for the contribution of the embedding are discussed in Section 4.6.

Actor. The actor has a federated *base* (two layers of 256 units) and a personal *head* (128 units) followed by three output heads: power (Gaussian, squashed by a sigmoid to $[P_{\min}, P_{\max}]$ with the corresponding log-density correction), channel (straight-through Gumbel-softmax over C categories with temperature annealed from 2.0 to 0.5, so that the one-hot vector that is executed is the one stored and given to the critic) and airtime budget (Dirichlet with concentrations softplus$(\cdot) + 1$). The critics are double-Q networks with two hidden layers of 256 units. The updates are the standard SAC updates,

$$\mathcal{L}_Q = \mathbb{E}\Big[\big(Q(s,a) - r - \gamma(\min_j \bar{Q}_j(s',a') - \alpha_{\mathrm{ent}} \log \pi(a'|s'))\big)^2\Big], \tag{6}$$

$$\mathcal{L}_\pi = \mathbb{E}\big[\alpha_{\mathrm{ent}} \log \pi(a|s) - \min_j Q_j(s,a)\big] + \tfrac{\mu}{2}\|\theta_{\mathrm{base}} - \theta_{\mathrm{anchor}}\|^2, \tag{7}$$

where α_{ent} is the entropy temperature (initialised to 0.05, automatically tuned towards a target entropy of -3) and the last term is the FedProx proximal term towards the last received base (active only in the two hierarchical variants). One gradient update of all agents is made after every environment step (update-to-data ratio one).

Starting from the incumbent configuration. In an operating venue the network already has a configuration (channel plan, power, equal airtime). All learned agents, including the learned baselines, are initialised close to it: the channel head is biased towards the AP's incumbent channel (round-robin plan), power starts at 14 dBm, the airtime split at $(frac13, \frac{1}{3}, \frac{1}{3})$, and the output layers are scaled down so that the initial policy depends little on the state. Exploration during the warm-up episodes is the noise of this initial stochastic policy around the incumbent configuration, not uniformly random actions; learning can then only depart from the incumbent where it finds an improvement. The one exception to the parametrisation is the single shared agent of LSTM-DRL, which has no AP-specific parameters and therefore outputs its channel as an offset from the AP's incumbent channel.

3.5. Bidirectional Joint Gradient

The predictor is trained on both the forecasting error and the actors' objective,

$$\mathcal{L}_{\text{pred}} = \underbrace{\frac{1}{N\Delta}\sum_{i,k}\left(\hat{L}_{i,k} - L^{*}_{i,k}\right)^{2}}_{\text{forecast error}} + \eta\,\frac{\mathcal{L}_{\pi}(\phi)}{|Q|+1}, \qquad \eta = 0.10, \tag{8}$$

where $\mathcal{L}_{\pi}(\phi)$ is Eq. (7) evaluated on states rebuilt from the *current* predictor embedding of stored samples (so the gradient $\partial\mathcal{L}_{\pi}/\partial\phi$ reaches ST-GCAT

through $\mathbf{Z}_{\text{fused}}$), and $\overline{|Q|}$ is the mean magnitude of the critic values in the batch (treated as a constant). Normalising by $\overline{|Q|}+1$ makes η independent of the reward scale; without it, the actor term dominates the gradient of the predictor. Only ϕ is updated in this step; the actor and critic parameters are updated by Eqs. (6)–(7). With $\eta = 0$ (ablation A1) the predictor is still trained, on the forecast error alone. States stored in the replay buffers contain the embedding of the time of storage; they are not recomputed when the predictor changes.

3.6. Module 3: HierFed-KD

Only the actor base layers ($797\cdot256 + 256 + 256\cdot256 + 256 = 270{,}080$ parameters) are federated; heads and critics stay local.

Level 1: zone aggregation. Every $T_{\text{local}} = 100$ environment steps the controller of zone z replaces the base of each AP by the quality-weighted average

$$\theta_z^{\text{zone}} = \sum_{i\in\mathcal{Z}_z} w_i\,\theta_i, \qquad w_i = \frac{\exp(\bar{r}_i/\tau_w)}{\sum_{j\in\mathcal{Z}_z}\exp(\bar{r}_j/\tau_w)}, \quad \tau_w = 0.1, \tag{9}$$

where $\bar{r}_i$ is the exponential moving average of the (unscaled) environment reward of AP i.

Level 2: proximal federated distillation across zones. Every $T_{\text{global}} = 500$ steps the zones do *not* exchange weights. Instead, all agents evaluate their actor on a public reference set $\mathcal{S}_{\text{ref}}$ of $M = 100$ states and report soft outputs: channel probabilities $\pi^{\text{ch}} \in \Delta^C$, mean airtime budget $\mathbf{w} \in \Delta^3$ and normalised power. A zone controller averages the outputs of its APs (the *zone teacher*) and the venue controller averages the zone teachers into a consensus $(\bar{\pi}^{\text{ch}}, \bar{\mathbf{w}}, \bar{p})$; this average is the closed-form minimiser of $\frac{1}{Z}\sum_z[\mathrm{KL}(\pi_z^{\text{ch}}\|\pi^{\text{ch}}) +$

$\|\mathbf{w}_z - \mathbf{w}\|^2 + \|p_z - p\|^2]$. The consensus is returned to the zones, and every agent distils its base layers towards it with a proximal term that keeps it close to its own model,

$$\min_{\theta_i} \ \mathrm{KL}\big(\bar{\pi}^{\mathrm{ch}} \,\|\, \pi_i^{\mathrm{ch}}\big) + \|\bar{\mathbf{w}} - \mathbf{w}_i\|^2 + \|\bar{p} - p_i\|^2 + \frac{\mu}{2}\|\theta_i - \theta_i^{\mathrm{before}}\|^2, \qquad \mu = 0.01, \tag{10}$$

solved with 20 Adam steps (learning rate $5 \cdot 10^{-4}$). The reference set contains states from one rollout of an independent proxy venue under random actions, observed through the current predictor; it does *not* contain any agent's own transitions, and it is refreshed every five global rounds.

The alternative that exchanges weights at this level (FedProx averaging of the zone models, $\theta^* = (\sum_z \theta_z + \mu\theta_{\mathrm{prev}})/(Z + \mu)$) is evaluated as a comparator in Section 4.7.

Communication accounting. We count the bytes of every message (32-bit floats). Per global round the distillation exchange costs $2ZM(C{+}4) \cdot 4$ bytes between zones and venue controller, instead of $2Z{\cdot}270{,}080{\cdot}4$ bytes for weights; the one-off and refresh cost of shipping $\mathcal{S}_{\mathrm{ref}}$ ($M \cdot 797 \cdot 4$ bytes per zone and refresh) is added to the inter-zone total. Measured totals are reported in Table 11. Zone-level traffic between APs and their zone controller is the same as in weight-based schemes and is reported separately.

3.7. Deployment Mapping: Channel Switching and QoS Queues

Channel switching. The channel in the action is applied at the start of the interval and costs service. A switching AP announces the change with channel-switch announcements (CSA); clients that miss it re-associate. We model this as an outage of $t_{\mathrm{sw}} = 3$ s of the 300 s interval: the AP's users receive a $t_{\mathrm{sw}}/300$

smaller share of the interval and the packets queued during the outage see an extra delay of $t_{\mathrm{sw}}/2$ on average, which enters the latency, throughput, reward and QoS metrics. The value is an engineering assumption (CSA with a few beacon intervals plus re-association of non-compliant clients); Section 4.8 reports results for $t_{\mathrm{sw}} \in \{0, 3, 10\}$ s and for a policy that was trained without any switching cost.

QoS queues. Commodity APs implement QoS with the four 802.11e EDCA access categories (AC_VO, AC_VI, AC_BE, AC_BK), each with its own contention parameters (AIFSN, $\mathrm{CW}_{\min}$, $\mathrm{CW}_{\max}$, TXOP limit). We map VoIP, video and best-effort to AC_VO, AC_VI and AC_BE. EDCA provides *relative priority*, not a bandwidth split, so the simplex action $\mathbf{w}_i$ is interpreted as an *airtime budget per access category* that is enforced on top of EDCA with the per-AC airtime quota (airtime-fairness / airtime-weight) scheduler available in current AP software, while the EDCA parameters stay at the standard values. To make the action implementable we (i) quantise $\mathbf{w}_i$ to multiples of 1/16 with a minimum of 1/16 per category (largest-remainder rounding), which is the granularity of integer airtime weights, and (ii) treat the budgets as limits that are work-conserving: unused airtime of an absent or satisfied category is redistributed to the others. The simulator applies exactly this quantisation and redistribution. Cross-vendor differences in how airtime quotas are exposed, and the presence of a fourth (background) category, are outside the scope of the evaluation; they only change how the three numbers are programmed, not the decision problem.

Algorithm 1 STAG-FedSAC training (one environment step shown; all APs act simultaneously).

Initialise ST-GCAT ϕ; for every AP i: actor π_i, critics Q_i, targets $\bar{Q}_i$, buffer $\mathcal{B}_i$, $\lambda_i = 0$.
for episode $= 1, \ldots, E$ **do**
    $o \leftarrow$ env.reset(); clear history
    for step $t = 1, \ldots, 12$ **do**
        Build $\mathbf{H}, \mathbf{S}, \mathbf{A}$; $\mathbf{Z}_{\text{fused}}, \hat{L} \leftarrow$ ST-GCAT$(\mathbf{H}, \mathbf{S}, \mathbf{A})$
        For every AP: $\mathbf{s}_i \leftarrow$ Eq. (5); $\mathbf{a}_i \sim \pi_i(\cdot|\mathbf{s}_i)$ (random during warm-up)
        $o', \{r_i\}, \text{done} \leftarrow$ env.step$(\{\mathbf{a}_i\})$
        $\mathcal{B}_i \leftarrow \mathcal{B}_i \cup \{(\mathbf{s}_i, \mathbf{a}_i, R_{\text{s}}(r_i + \delta_p b_i^{\text{pred}} - \text{penalty}_i), \mathbf{s}_i')\}$; update λ_i (Eq. 2)
        Train ST-GCAT on stored samples (Eq. 8; with the actor term only after warm-up)
        if step $\geq T_{\text{warmup}}$ **then**
            SAC update of critics, actors, temperature of all agents (Eqs. 6–7)
        end if
        if step $\bmod T_{\text{local}} = 0$ **then**
            Zone aggregation (Eq. 9)
        end if
        if step $\bmod T_{\text{global}} = 0$ **then**
            Proximal federated distillation (Eq. 10)
        end if
    end for
    Attach true multi-step targets to the episode's predictor samples
end for

4. Experiments

4.1. Simulator

All experiments use a Python simulator of a transit terminal with $N = 30$ APs on a 6×5 grid (30 m spacing, 2 m position jitter) in $Z = 3$ zones ($i \bmod 3$). It is a quasi-static model: all quantities are computed once per 5-minute control interval, for the users present in that interval. Table 5 lists the parameters. The simulator was extended for this revision with an explicit contention model, a channel-switching cost and the EDCA-style airtime mapping of Section 3.7.

Table 5: Simulator parameters.

Item	**Value**
Venue	30 APs, 6×5 grid, 30 m spacing, 3 zones
Path loss / noise / carrier-sense threshold	$46.4 + 30 \log_{10} d$ dB / -95 dBm / -82 dBm
Channels, bandwidth	11 orthogonal, 20 MHz
PHY rate	$\min\{156, 0.75 \cdot 20 \log_2(1 + \text{SINR})\}$ Mbps
MAC	Bianchi DCF, 16-MPDU A-MPDU, $\text{CW}_{\min}$ 16, 6 stages; 25
Offered rate VoIP / video / best-effort	0.1 / 3 / 1 Mbps per user (mix 20 / 30 / 50%)
SINR threshold VoIP / video / best-effort	10 / 15 / 5 dB; latency SLA 50 / 100 / 500 ms
Users	500–1000 at episode start; surge users stay two intervals (a
Channel-switch outage	3 s per switch (sensitivity: 0, 10 s)
Airtime budget granularity	1/16, minimum 1/16 per access category
AP power model	6 W static + transmit power / 0.3

Radio. Path loss is $PL(d) = 46.4 + 30\log_{10} d$ dB (5 GHz, indoor), noise -95 dBm, and $C = 11$ orthogonal 20 MHz channels (adjacent-channel leakage is not modelled). A user associates to its nearest AP. The PHY rate of user u is $R_u = \min\{156,\ 0.75 \cdot 20\log_2(1 + \mathrm{SINR}_u)\}$ Mbps (2 spatial streams, 256-QAM cap), with at least 1 Mbps.

MAC contention. Two APs that can hear each other (received power ≥ -82 dBm in either direction) and share a channel form a *carrier-sense domain*: they time-share the medium, each receiving the airtime fraction $n_i / \sum_{j \in \mathcal{D}_i} n_j$ of the domain's users, and they do not interfere with each other. Only co-channel APs outside the domain add interference to the SINR (hidden terminals are not modelled). With $m_i = \max(1, \lfloor 0.25 \sum_{j \in \mathcal{D}_i} n_j \rceil)$ backlogged stations in the domain, the fraction of airtime that carries payload is the saturation efficiency $\eta(m_i)$ of the IEEE 802.11 DCF obtained from Bianchi's fixed-point model [27] (802.11ac timings, 16-MPDU A-MPDUs, $\mathrm{CW}_{\min} = 16$, 6 back-off stages; $\eta(1) = 0.87$, $\eta(10) = 0.70$, $\eta(50) = 0.56$, $\eta(100) = 0.50$). The airtime available to AP i is $\rho_i^{\mathrm{avail}} = \eta(m_i)\, n_i / \sum_{j \in \mathcal{D}_i} n_j$. User u of class c is offered a constant rate d_c (0.1, 3 and 1 Mbps for VoIP, video and best-effort) and obtains the part of it that fits into its class's airtime budget (Section 3.7, with redistribution of unused airtime).

Latency. The per-class latency is $5 + s_i\,[1 + \varrho/(2(1-\varrho))]$ ms, an M/G/1-type delay with the domain service time $s_i = (\text{A-MPDU payload time})/\eta(m_i)$ and $\varrho = \min(0.98,\ \text{airtime needed by the class/airtime granted to it})$; if the class is overloaded ($\varrho > 0.98$) the delay rises linearly to the 500 ms buffer limit as ϱ grows to 1.48. A channel switch adds half of the outage time to the delay of the AP's users (Section 3.7). Latency in this model is therefore never a

constant: it depends on the number of contending stations, the class budget and the load.

Users and surges. At the start of an episode between 500 and 1000 users are placed uniformly in the venue (20% VoIP, 30% video, 50% best-effort) and move by a 2 m random walk per interval. An episode is 12 intervals (1 hour) and contains 3–7 timetable events (departure or arrival, 50–300 passengers, random zone and time). Around each event passengers join the venue for two intervals, at most 100 per interval, producing the load surges that the schedule announces. Timetable data are synthetic.

Metrics. Reward is the unscaled Eq. (1) without b^{pred} averaged over APs and intervals. *Throughput* is the mean per-user throughput (Mbps), *latency* the mean per-user latency, *JFI* Jain's index of the user satisfaction ratios ς_u over all users of the venue, *QoS satisfaction* the fraction of users that meet their class requirement (SINR, latency and $\varsigma_u \geq 0.5$), *energy efficiency* the total throughput divided by the power drawn by all APs (6 W static plus transmit power divided by a PA efficiency of 0.3), and *switches* the fraction of APs that change channel per interval. Forecast quality is measured by RMSE and MAE over all $\Delta = 6$ horizons (and for $h = 1$ only), MAPE on loads above 0.05, and *Surge-RMSE*: the RMSE over AP/interval pairs that lie within one interval of a scheduled event with at least 100 passengers in the AP's zone (Normal-RMSE: all other pairs).

4.2. Compared Methods

- **SSF** (strongest signal first, static): users keep their strongest AP, fixed round-robin channel plan, mid power, equal airtime split.

- **LLF** (least loaded first, reactive): lowers power on overloaded APs and raises it on idle ones, splits airtime by the observed QoS mix, fixed channel plan.

- **LSTM-DRL**: an LSTM forecasts the next loads of each AP from its own history (no graph, no schedule); one DDPG agent *shared by all APs* acts on the local features, the QoS mix and the scalar one-step forecast. No federation.

- **FedDDPG** (adapted from [2]): one DDPG agent per AP, flat FedAvg of actor and critic every T_{local} steps, no forecast, no schedule.

- **STAG-FedSAC** and its ablations: **A1** no joint gradient ($\eta = 0$; predictor still trained on the forecast error), **A2** no schedule (the schedule tensor and the schedule part of the state are zeroed), **A3** no embedding ($\mathbf{Z}_{\text{fused}}$ in the state is zeroed).

- **Federation variants** (Section 4.7): isolated agents, flat FedAvg, zone FedAvg only (Level 1), zone FedAvg with weight-based global FedProx, and a pooled-experience learner.

- **Persistence**: a forecast reference that predicts the current load for all horizons (with SSF control).

All learned methods are trained with the same protocol (same simulator, the same reward plus Lagrangian penalty and reward scale, 200 warm-up episodes of exploration around the incumbent configuration, replay buffer, one gradient update per environment step, same number of episodes) and are evaluated on the same held-out episodes. The two DDPG baselines use

Gaussian exploration noise $\sigma = 0.2$ and encode the channel as one continuous output rounded to an index. We did not tune any method separately; hyperparameters are shared wherever the architectures allow it.

4.3. Protocol and Hyperparameters

Every learned method is trained for 1000 episodes (the first 200 are warm-up) and evaluated on 20 held-out test episodes whose seeds are disjoint from the training and validation seeds and identical for all methods (Table 7: seeds 0–2). The first version of this paper stated 2000 training episodes; we use 1000 here because the learning curves flatten well before that point (Fig. 2) and because every method had to be re-run with the extended simulator. The federation variants of Section 4.7 use seeds 0–1 (STAG-FedSAC: 0–2), the robustness runs trained without contention or without switching cost seeds 0–1, the reward-weight and the η sensitivity runs seed 0 (weights) and seeds 0–2 ($\eta = 0.01$). The number of seeds is given in every table. A seed fixes the network initialisation, the simulator layout and the training episodes; the metrics of a run are the mean over its 20 test episodes, and the tables show the mean and standard deviation of these run-level values over seeds. A five-episode validation during training (seeds disjoint from the test seeds) produces the learning curves and was used for the only tuned value, κ.

4.4. Main Results

Learned control versus the static plan. The best learned controllers do not improve on the incumbent configuration. SSF reaches a reward of 0.181, STAG-FedSAC 0.141 ± 0.008 (mean $\pm$ standard deviation over 3 seeds); the paired difference is -0.041 with a 95% bootstrap interval of $[-0.049, -0.033]$,

Table 6: Hyperparameters (identical for all learned methods where applicable); the incumbent configuration is round-robin channels, 14 dBm, equal airtime.

Parameter	**Symbol**	**Value**
Environment		
APs / channels / zones	N, C, Z	30, 11, 3
Control interval / episode		5 min / 12 intervals
History window / forecast horizon	T, Δ	12 (60 min) / 6 (30 min)
Channel-switch outage / airtime granularity	t_{sw} / EDCA levels	3 s / 1/16
Reward		
Throughput, latency, fairness, QoS, prediction	$\alpha, \beta, \gamma, \delta, \delta_p$	0.40, 0.25, 0.20, 0.05, 0.10
Reward scale for learning	R_{s}	10
ST-GCAT		
GAT / Transformer heads, layers, d_h	K, L, d_h	4, 2, 128
Predictor learning rate, batch	η_ϕ	5×10^{-4}, 16 samples
Joint gradient weight	η	0.10
SAC		
Actor / critic hidden layers		[256, 256] (+128 personal head)
Batch, replay size per AP		256, 24,000 transitions (all episo
Learning rate actor, critic, temperature		3×10^{-4}
Discount / soft target update	γ, τ	0.99 / 0.005
Initial temperature / target entropy		0.05 / -3
Gumbel-softmax temperature		$2.0 \rightarrow 0.5$ (linear)
Embedding scale in the state	κ	0.03 (chosen from $\{1, 0.125, 0.03$
HierFed-KD		
Zone / global round	T_ℓ, T_g	100 / 500 steps
Proximal weight, quality temperature	μ, τ_w	0.01, 0.1
Reference states, distillation steps, lr	M	100, 20, 5×10^{-4}
Constraint		
Utilisation limit, step, bound	$u_{\max}, \rho_\lambda, \lambda_{\max}$	0.9, 0.01, 1

Table 7: Held-out performance (20 test episodes per run; mean and standard deviation over training seeds). Bold: best. The reward is the unscaled Eq. (1) without b^{pred}. "Sw./AP/step" includes the initial reconfiguration from the random channels at the start of each episode, which every method pays equally.

Method	Seeds	Reward	Thrpt. (Mbps/user)	Lat. (ms)	JFI	QoS sat.	Mbps/W	Sw./AP/step
SSF	3	$0.181_{\pm 0.001}$	$1.31_{\pm 0.00}$	$124.6_{\pm 2.2}$	$\mathbf{0.985_{\pm 0.001}}$	$0.682_{\pm 0.002}$	$7.40_{\pm 0.03}$	$\mathbf{0.075_{\pm 0.000}}$
LLF	3	$0.156_{\pm 0.001}$	$1.22_{\pm 0.01}$	$145.0_{\pm 2.2}$	$0.967_{\pm 0.002}$	$0.638_{\pm 0.002}$	$6.85_{\pm 0.04}$	$\mathbf{0.075_{\pm 0.000}}$
LSTM-DRL	3	$0.108_{\pm 0.004}$	$1.11_{\pm 0.01}$	$198.9_{\pm 8.0}$	$0.928_{\pm 0.007}$	$0.577_{\pm 0.021}$	$6.07_{\pm 0.07}$	$0.668_{\pm 0.011}$
FedDDPG	3	$0.116_{\pm 0.004}$	$1.16_{\pm 0.01}$	$184.8_{\pm 7.2}$	$0.943_{\pm 0.005}$	$0.602_{\pm 0.010}$	$6.40_{\pm 0.08}$	$0.752_{\pm 0.031}$
A1: no joint gradient	3	$0.176_{\pm 0.009}$	$1.30_{\pm 0.03}$	$118.4_{\pm 9.8}$	$0.979_{\pm 0.008}$	$0.679_{\pm 0.017}$	$7.34_{\pm 0.16}$	$0.128_{\pm 0.011}$
A2: no schedule	3	$0.128_{\pm 0.010}$	$1.16_{\pm 0.03}$	$183.9_{\pm 14.3}$	$0.937_{\pm 0.007}$	$0.584_{\pm 0.019}$	$6.45_{\pm 0.18}$	$0.214_{\pm 0.013}$
A3: no embedding	3	$\mathbf{0.182_{\pm 0.006}}$	$\mathbf{1.32_{\pm 0.02}}$	$\mathbf{109.9_{\pm 7.0}}$	$0.985_{\pm 0.005}$	$\mathbf{0.689_{\pm 0.014}}$	$\mathbf{7.47_{\pm 0.13}}$	$0.153_{\pm 0.014}$
STAG-FedSAC	3	$0.141_{\pm 0.008}$	$1.19_{\pm 0.02}$	$170.6_{\pm 8.6}$	$0.952_{\pm 0.007}$	$0.614_{\pm 0.017}$	$6.65_{\pm 0.14}$	$0.165_{\pm 0.009}$

and all seeds have the same sign. With three seeds the exact sign-flip test cannot give a p-value below 0.25, so Table 8 reports effect sizes and intervals instead of significance. A perfect-information greedy optimiser that evaluates candidate configurations in the simulator (not a deployable controller; no switching cost) reaches 0.216 against 0.183 for SSF on 10 test episodes, i.e. +18%. There is therefore headroom above the incumbent, but within the training budget used here none of the learned controllers realises it. We do not claim that STAG-FedSAC outperforms a well-chosen static plan in this simulator.

Learned methods. STAG-FedSAC is clearly better than the two learned baselines (reward 0.141 against 0.116 for FedDDPG and 0.108 for LSTM-DRL; paired differences +0.024 and +0.032, intervals excluding zero). Both DDPG baselines start at the incumbent configuration like every other learned method, but learning moves them away from it: they change the channel of 0.752 (FedDDPG) and 0.668 (LSTM-DRL) APs per interval against 0.075 for the fixed

Table 8: Paired difference in reward between STAG-FedSAC and each other method over matched training seeds (positive: STAG-FedSAC better), 95% bootstrap interval of the mean difference, exact two-sided sign-flip p-value (its minimum is 2^{1-n} for n seeds) and its Bonferroni-adjusted value.

STAG-FedSAC vs.	Δ reward	95% CI	p	p_{Bonf}	Seeds
SSF	-0.041	[-0.049, -0.033]	0.250	1.000	3
LLF	-0.016	[-0.024, -0.007]	0.250	1.000	3
LSTM-DRL	+0.032	[+0.029, +0.038]	0.250	1.000	3
FedDDPG	+0.024	[+0.018, +0.034]	0.250	1.000	3
A1: no joint gradient	-0.035	[-0.045, -0.024]	0.250	1.000	3
A2: no schedule	+0.013	[-0.007, +0.023]	0.500	1.000	3
A3: no embedding	-0.041	[-0.048, -0.031]	0.250	1.000	3

plan, and each change costs service (Section 3.7). The SAC-based variants move far less (0.165 for STAG-FedSAC). LLF, the reactive heuristic, is below the static plan (0.156). We read this as evidence that, in this simulator, channel and power changes are expensive relative to what a one-hour episode lets a learner gain, and that the stability of the SAC-based agents (entropy regularisation, personal heads, proximal term) matters more than any of the other design choices. Figure 2 shows the trajectories: the DDPG baselines leave the incumbent level immediately after the warm-up and stay far below it, STAG-FedSAC declines more slowly, and the variant without the joint gradient stays close to the incumbent level throughout.

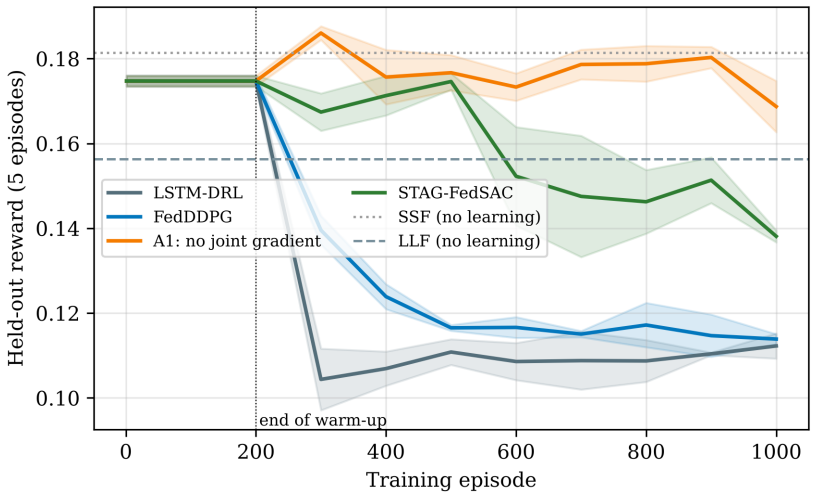


Figure 2: Reward on five held-out validation episodes during training (mean and standard deviation over seeds). During the first 200 episodes the actors only explore around the incumbent configuration (no gradient updates of the policy), so every learned method starts at the level of the incumbent plan; horizontal lines are the non-learning baselines.

Table 9: Load forecast error on the held-out episodes (load is normalised to $[0, 1]$; all $\Delta = 6$ horizons unless stated). Surge: AP/interval pairs within one interval of a scheduled event with at least 100 passengers in the AP's zone.

Forecaster	RMSE	MAE	RMSE (h=1)	Surge-RMSE	Normal-RMSE	MAPE (%)
Persistence	$0.186_{\pm 0.001}$	$0.130_{\pm 0.000}$	$0.096_{\pm 0.000}$	$0.266_{\pm 0.002}$	$0.162_{\pm 0.001}$	$27.8_{\pm 0.4}$
LSTM (temporal only)	$0.142_{\pm 0.000}$	$0.107_{\pm 0.002}$	$0.075_{\pm 0.001}$	$0.213_{\pm 0.010}$	$0.118_{\pm 0.004}$	$24.4_{\pm 1.0}$
ST-GCAT, no schedule (A2)	$0.175_{\pm 0.003}$	$0.135_{\pm 0.004}$	$0.159_{\pm 0.004}$	$0.243_{\pm 0.005}$	$0.152_{\pm 0.005}$	$30.2_{\pm 1.7}$
ST-GCAT, $\eta = 0$ (A1)	$0.121_{\pm 0.000}$	$0.091_{\pm 0.001}$	$0.126_{\pm 0.003}$	$0.163_{\pm 0.009}$	$0.111_{\pm 0.002}$	$20.4_{\pm 0.7}$
STAG-FedSAC	$0.127_{\pm 0.000}$	$0.097_{\pm 0.001}$	$0.132_{\pm 0.000}$	$0.169_{\pm 0.006}$	$0.116_{\pm 0.002}$	$21.3_{\pm 0.6}$

4.5. Forecast Quality

Forecasting is where the schedule information pays off, at the horizons that matter for proactive control. Over all six horizons STAG-FedSAC's ST-GCAT reaches an RMSE of 0.127 against 0.186 for persistence (-32%), 0.142 for the temporal-only LSTM (-11%) and 0.175 for ST-GCAT without schedule input (-27%). The gain is larger in the surge windows that the timetable announces: Surge-RMSE is 0.169 against 0.266, 0.213 and 0.243 (-36%, -21%, -31%), while the improvement over the LSTM outside surges is small (-1%). Without the schedule, the graph-attention predictor is *worse* than the LSTM (0.175 against 0.142), which indicates that its advantage comes from the cross-attention on the timetable and not from the graph or Transformer layers alone. The variants trained without the joint gradient (A1) and without the embedding in the policy (A3) have a slightly lower error than the full model (0.121 and 0.121 against 0.127), so the joint gradient does not improve forecasts in our implementation (Section 4.6). The advantage is not uniform over horizons: at the one-step horizon ST-GCAT is *less* accurate than the simple forecasters (RMSE 0.132 against 0.096 for persistence and 0.075 for the LSTM), and the gain appears at the longer horizons, where the timetable carries information that the history does not. For very short-term prediction a simple forecaster is preferable. Two further caveats apply. First, the timetable of the simulator is exact and generated by the same process that creates the surges; real timetables contain delays and cancellations, so the gain reported here is an upper bound for a deployment. Second, the load being forecast is the number of users attached to an AP, which the actions do not affect.

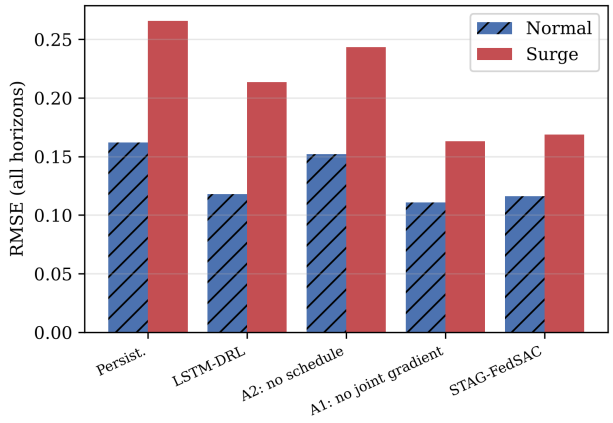


Figure 3: Forecast RMSE on surge and non-surge AP/interval pairs.

4.6. Ablation of the Contributions

We remove one element at a time (Tables 7 and 9).

A2: no schedule. Removing the schedule worsens both forecasts (RMSE 0.175 against 0.127) and control (reward 0.128 against 0.141, latency 183.9 against 170.6 ms); the interval of the control difference ($[-0.007, +0.023]$) includes zero for the reward with three seeds, so only the forecasting effect is conclusive.

A3: no embedding. Zeroing the embedding in the policy state does *not* hurt; the reward is 0.182 against 0.141 with the embedding. At the embedding scale we use, the policy gets no measurable control benefit from $\mathbf{Z}_{\text{fused}}$ beyond what the schedule features and local counters already provide. We therefore cannot claim that feeding the full embedding into the SAC state improves control; we only show that doing so is possible and that it does not degrade the forecasts. Larger embedding scales reduced the validation reward in our

pilot runs (Section 3.4), which suggests that the stale embeddings in the replay buffer, not the information content, limit its usefulness.

A1: no joint gradient. Removing the bridge ($\eta = 0$) *improves* the reward (0.176 against 0.141) and slightly improves the forecast (0.121 against 0.127). The actor term of Eq. (8) moves the embedding while the replay buffers hold embeddings computed earlier, which makes the critic's inputs non-stationary. We could not show a benefit of the joint gradient, and we report this contribution as a negative result for the weight tested here ($\eta = 0.1$). A sweep over the weight confirms this (Table 10): with $\eta = 0$, 0.01 and 0.1 the reward is 0.176, 0.146 and 0.141 and the forecast RMSE 0.121, 0.122 and 0.127; no tested value improved on $\eta = 0$.

Table 10: Weight η of the joint gradient (mean and standard deviation over seeds). $\eta = 0$ is ablation A1.

η	Seeds	Reward	Lat. (ms)	RMSE	Surge-RMSE
0	3	$0.176_{\pm 0.009}$	$118.4_{\pm 9.8}$	$0.121_{\pm 0.000}$	$0.163_{\pm 0.009}$
0.01	3	$0.146_{\pm 0.005}$	$164.0_{\pm 13.2}$	$0.122_{\pm 0.001}$	$0.167_{\pm 0.007}$
0.1	3	$0.141_{\pm 0.008}$	$170.6_{\pm 8.6}$	$0.127_{\pm 0.000}$	$0.169_{\pm 0.006}$

4.7. What Does Federation Buy?

All variants use the same agents, reward, protocol and seeds and differ only in how base layers are shared; Table 11 lists their held-out performance and the measured traffic of the whole training run.

Table 11: Federation schemes for the same agents. Traffic is measured during training, summed over all APs and rounds (MB); inter-zone traffic includes the reference states shipped for distillation.

Scheme	Seeds	Reward	Lat. (ms)	JFI	QoS sat.	Intra-zone (MB)	Inter-zone (MB)
Isolated (no federation)	2	$0.121_{\pm 0.004}$	$184.6_{\pm 2.8}$	$0.935_{\pm 0.009}$	$0.584_{\pm 0.015}$	0.0	0.0
Flat FedAvg	2	$0.141_{\pm 0.006}$	$162.8_{\pm 2.9}$	$0.954_{\pm 0.002}$	$0.615_{\pm 0.000}$	0.0	7778.3
Zone FedAvg only	2	$0.132_{\pm 0.005}$	$176.9_{\pm 9.0}$	$0.946_{\pm 0.010}$	$0.606_{\pm 0.021}$	7778.3	0.0
Hier. FedAvg+FedProx (weights)	2	$0.146_{\pm 0.005}$	$156.9_{\pm 9.3}$	$0.959_{\pm 0.009}$	$0.637_{\pm 0.010}$	7778.3	155.6
STAG-FedSAC	3	$0.141_{\pm 0.008}$	$170.6_{\pm 8.6}$	$0.952_{\pm 0.007}$	$0.614_{\pm 0.017}$	7785.5	4.5
Pooled experience (centralised)	2	$0.100_{\pm 0.013}$	$213.0_{\pm 7.0}$	$0.902_{\pm 0.028}$	$0.534_{\pm 0.028}$	0.0	0.0

Effect of federation. Yes, moderately. Isolated agents obtain a reward of 0.121 (latency 184.6 ms); zone-level averaging alone 0.132, flat FedAvg 0.141, weight-based hierarchical federation 0.146 and the proposed distillation-based federation 0.141. All federated variants are above the isolated agents, by about 0.01–0.03, which supports sharing what the agents learn. The differences *among* the federated variants are of the order of the seed-to-seed variation (two to three seeds), so we cannot rank them and make no claim that hierarchical distillation is more accurate than flat averaging or than weight-based hierarchical federation.

Pooled experience is not an upper bound. The comparator in which each agent samples its minibatches from the experience of *all* APs, which violates the privacy motivation, obtains only 0.100, below isolated learning. A plausible reason is that the policies are AP-specific (different channels, loads and neighbourhoods), so transitions of other APs are off-distribution for an agent's own critic. We therefore cannot use it to quantify a "price of privacy" in this setting.

Communication. Over a whole run, flat FedAvg between all APs and a central server moves 7778.30 MB over the wide-area link. Weight-based hierarchical federation reduces this to 155.57 MB (only the zone models travel between zones and venue controller); the distillation-based Level 2 of HierFed-KD to 4.55 MB, of which 3.8 MB are the reference states shipped to the zones and 0.72 MB the soft outputs, i.e. 34× less than weight-based hierarchical federation and 1711× less than flat FedAvg, at a similar reward. Traffic inside zones is unchanged (7785.5 MB per run) because Level 1 exchanges weights in every hierarchical variant. These ratios replace the single "11.5×" figure of the first version, which was not measured; they depend on the choices made here ($M = 100$ reference states, refresh every five global rounds, $T_{\text{global}} = 500$) and count only payload bytes.

4.8. Robustness to the Simulator Assumptions

Table 12: Reward / latency (ms) of the same trained policies evaluated under different assumptions. A policy trained without contention ("w/o MAC") or without switching cost is evaluated in the full simulator in the first column.

Policy	Full MAC, 3 s outage	No contention	No switch cost	10 s outage
SSF (trained w/ MAC)	$0.181_{\pm 0.001}$ / $125_{\pm 2}$	$0.222_{\pm 0.001}$ / $70_{\pm 1}$	$0.184_{\pm 0.001}$ / $124_{\pm 2}$	$0.177_{\pm 0.001}$ / $135_{\pm 2}$
LLF (trained w/ MAC)	$0.156_{\pm 0.001}$ / $145_{\pm 2}$	$0.179_{\pm 0.002}$ / $117_{\pm 3}$	$0.159_{\pm 0.001}$ / $144_{\pm 2}$	$0.152_{\pm 0.001}$ / $155_{\pm 2}$
FedDDPG (trained w/ MAC)	$0.116_{\pm 0.004}$ / $185_{\pm 7}$	$0.174_{\pm 0.002}$ / $106_{\pm 4}$	$0.136_{\pm 0.004}$ / $176_{\pm 8}$	$0.084_{\pm 0.003}$ / $270_{\pm 7}$
STAG-FedSAC (trained w/ MAC)	$0.141_{\pm 0.008}$ / $171_{\pm 9}$	$0.200_{\pm 0.005}$ / $93_{\pm 4}$	$0.146_{\pm 0.007}$ / $168_{\pm 8}$	$0.134_{\pm 0.008}$ / $190_{\pm 10}$
FedDDPG (trained w/o MAC)	$0.060_{\pm 0.000}$ / $257_{\pm 16}$	$0.165_{\pm 0.000}$ / $123_{\pm 9}$	$0.073_{\pm 0.001}$ / $251_{\pm 16}$	$0.041_{\pm 0.004}$ / $327_{\pm 4}$
STAG-FedSAC (trained w/o MAC)	$0.105_{\pm 0.010}$ / $217_{\pm 22}$	$0.178_{\pm 0.002}$ / $121_{\pm 5}$	$0.109_{\pm 0.009}$ / $216_{\pm 20}$	$0.099_{\pm 0.008}$ / $238_{\pm 17}$
STAG-FedSAC (trained w/o switch cost)	$0.134_{\pm 0.015}$ / $174_{\pm 22}$	$0.192_{\pm 0.007}$ / $100_{\pm 11}$	$0.139_{\pm 0.015}$ / $172_{\pm 21}$	$0.123_{\pm 0.016}$ / $202_{\pm 21}$

MAC contention. Evaluated without contention, the very same policies obtain a much lower latency: SSF 70.3 ms instead of 124.6 ms and STAG-FedSAC 92.5 ms instead of 170.6 ms, i.e. a factor of about 1.8 in both cases,

and a higher reward (0.200 against 0.141 for STAG-FedSAC). A simulator that ignores contention therefore understates latency by roughly 45%, and the "clean" latency of the first version was an artefact of that omission. Policies that were *trained* without contention and evaluated with it are clearly worse than policies trained with it (STAG-FedSAC 0.105 against 0.141; FedDDPG 0.060 against 0.116; two training seeds for the former), so contention-unaware planning is not only optimistic but also leads to worse configurations.

Channel-switching cost. Varying the outage in the evaluation from 0 to 3 to 10 s changes the reward of STAG-FedSAC from 0.146 to 0.141 to 0.134, and its latency from 167.5 to 170.6 to 189.6 ms. The cost matters most for methods that switch often: FedDDPG, which changes the channel of about three quarters of the APs per interval, falls from 0.136 to 0.084 between 0 and 10 s, whereas SSF, which does not switch after the first interval, is almost unaffected. A policy trained without any switching cost and evaluated with the 3 s cost reaches 0.134 against 0.141 for the policy trained with it (two seeds); this difference is within the seed-to-seed variation, so we cannot show that training with the cost helps.

4.9. Sensitivity to the Reward Weights

The reward weights are design choices (Section 3.2). To see whether the conclusions depend on them, we retrained STAG-FedSAC and FedDDPG with three other weightings and evaluated SSF with the same weights (Table 13; the first row repeats the default weights for the same seed). The ordering of the methods is the same for all four weightings: SSF has the highest reward, STAG-FedSAC is second and FedDDPG third. STAG-FedSAC also

Table 13: Sensitivity to the reward weights (one training seed per cell; reward is computed with the weights used in training, so rewards are not comparable across rows; latency and JFI are).

	SSF			FedDDPG			STAG-FedSAC		
Weights ($\alpha/\beta/\gamma/\delta$)	Reward	Lat.	JFI	Reward	Lat.	JFI	Reward	Lat.	JFI
Default (0.40/0.25/0.20/0.05)	0.182	124	0.986	0.114	189	0.941	0.133	180	0.947
Equal weights (0.25/0.25/0.25/0.25)	0.106	124	0.986	0.046	165	0.935	0.059	166	0.952
Throughput-heavy (0.55/0.15/0.20/0.10)	0.308	124	0.986	0.226	196	0.928	0.250	188	0.942
Latency-heavy (0.20/0.50/0.20/0.10)	-0.106	124	0.986	-0.182	186	0.936	-0.122	135	0.972

has a higher JFI than FedDDPG in every row and a lower latency in three of the four rows (the equal-weights row is a tie within 2 ms), the largest gap being under the latency-heavy weights. Each cell is a single training seed, so the table indicates robustness of the qualitative picture, not precise differences.

5. Discussion and Limitations

What the evidence supports. (i) Conditioning a graph-attention forecaster on the timetable reduces load-forecast error substantially over the six-interval horizon, in particular in the announced surge windows (Section 4.5), although at the one-step horizon simple forecasters are more accurate; the effect is robust over seeds and is not obtained by the graph or Transformer layers without the schedule. (ii) Per-AP SAC agents started from the incumbent configuration remain close to it, whereas the DDPG-based learned baselines trained with the same protocol drift away and lose reward, mostly through channel switching (Section 4.4). (iii) Ignoring MAC contention and channel-switching cost changes the evaluation materially (Section 4.8), which is a

caution for simulator-only studies of Wi-Fi resource management, including our first version.

What it does not support. The learned controllers do not outperform the static incumbent plan in the simulator, although a perfect-information optimiser shows that headroom exists. Neither feeding the ST-GCAT embedding into the policy state (Contribution 2) nor the bidirectional joint gradient (Contribution 3) improves control or forecasting in our experiments; the joint gradient with $\eta = 0.1$ made both slightly worse. These two components should be regarded as unproven. The reward weights are design choices, not derived quantities (Section 4.9).

Simulator. The simulator is quasi-static (one evaluation per 5-minute interval) and is not calibrated to measurements; no measured traces are used. Hidden terminals, adjacent-channel interference, frame-level dynamics, rate adaptation and client roaming (users always associate to the nearest AP) are not modelled; contention uses the saturation model of Bianchi with a fixed share of backlogged stations. The timetable is synthetic and exact. The channel-switching outage (3 s) and the EDCA airtime mapping are engineering assumptions that we did not validate on hardware; the fourth EDCA category and vendor-specific airtime-quota interfaces are not covered. Results are for one venue layout with 30 APs.

Training and statistics. Each run uses 1000 episodes (200 of exploration around the incumbent configuration); longer training, other hyperparameters or other exploration schemes might change the control results. The number of seeds is small (three for the main comparison), so we report effect

sizes and bootstrap intervals; no result of the paper depends on a significance threshold. The design choices made while developing the method (reward scale, initial entropy temperature, update-to-data ratio, initialisation at the incumbent configuration, embedding scale κ) were selected on validation episodes of one seed, mostly with variants of STAG-FedSAC; the baselines were not tuned separately, which favours the proposed method over the baselines. All learned methods start from the incumbent configuration, which is an inductive bias; it favours stability and may not be available when no sensible configuration exists.

Privacy and federation. Federation is used for data minimisation, not for a formal guarantee: neither differential privacy nor secure aggregation is applied, and model parameters and distilled outputs can leak information. The predictor is centralised at the venue controller and consumes AP-level aggregates. The public reference set is generated from an independent proxy venue, which assumes that such a simulator is available to all parties; its refresh traffic is included in the measured communication.

Future work. Packet-level simulation (e.g. ns-3 [28]) with hidden terminals and rate adaptation; calibration and validation on measured venue traces such as [29]; recomputing embeddings for stored transitions (or learning with fresh embeddings) to make the joint gradient usable; longer training and safe-improvement schemes that let the learner depart from the incumbent only where it has evidence of gains; delayed or uncertain timetables; and differential privacy or secure aggregation for the federation.

6. Conclusion

We studied schedule-driven WiFi resource management in transit venues with a schedule-conditioned graph-attention forecaster, per-AP Soft Actor-Critic agents and a two-level federation, in a simulator that, unlike the first version of this paper, models MAC contention, channel-switching cost and a realisable QoS mapping. The clearest result is on forecasting: conditioning on the timetable lowers the RMSE by 32% relative to persistence and 27% relative to the same predictor without the schedule, and by 31% in surge windows. On control, the learned federated agents are stable and far better than learned DDPG baselines, but they do not outperform the static incumbent configuration in our simulator, and we could not show a benefit of feeding the embedding to the policy or of the joint gradient. Federation improves on isolated agents (reward 0.141 against 0.121), and exchanging soft policy outputs between zones needs 1711× less wide-area traffic than flat averaging and 34× less than weight-based hierarchical federation at a similar reward; the reward differences among the federated variants are within seed noise. The conclusions are limited to simulation; the open questions are listed above. All numbers in this paper are generated by the code and run records in the public repository (`https://github.com/vinhqdang/STAG_FedSAC`); no trained checkpoints are distributed.

Declaration of Competing Interest

The authors declare no competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

No measured data are used. The simulator, the synthetic timetables, the run records of all experiments and the analysis scripts are included in the public repository `https://github.com/vinhqdang/STAG_FedSAC`.

Acknowledgements

The author thanks the British University Vietnam for access to the NVIDIA RTX 5000 Ada GPU used in the development of the first version of this work. The experiments reported in this revision were run on Google Colab GPU instances (NVIDIA T4).

Funding

This research received no external funding.

Ethics declaration

Not applicable.

References

[1] G. Kougioumtzidis, V. K. Poulkov, P. I. Lazaridis, Z. D. Zaharis, Mobile network traffic prediction using temporal fusion transformer, IEEE Transactions on Artificial Intelligence (2025).

[2] H. Du, Z. Li, D. Niyato, J. Kang, Z. Xiong, D. I. Kim, Enabling AI-generated content (AIGC) services in wireless edge networks, IEEE

Wireless Communications 31 (1) (2024) 16–24. doi:10.1109/MWC.011.2200494.

[3] X. Pang, J. Hu, P. Sun, J. Ren, Z. Wang, When federated learning meets knowledge distillation, IEEE Wireless Communications 31 (5) (2024) 208–214.

[4] A. Alauthman, T. Shraa, Deep reinforcement learning-driven dynamic spectrum access in dense Wi-Fi environments, IEEE Access (2025).

[5] J. Yu, L. Liang, C. Guo, Z. Guo, S. Jin, G. Y. Li, Heterogeneous multi-agent reinforcement learning for distributed channel access in WLANs, IEEE Transactions on Cognitive Communications and Networking (2026).

[6] S. S. Shaabanzadeh, J. Sánchez-González, A spatio-temporal prediction methodology based on deep learning and real Wi-Fi measurements, Computer Networks 250 (2024) 110569. doi:10.1016/j.comnet.2024.110569.

[7] Z. Cai, C. Tan, J. Zhang, L. Zhu, Y. Feng, DBSTGNN-Att: Dual branch spatio-temporal graph neural network with an attention mechanism for cellular network traffic prediction, Applied Sciences 14 (5) (2024) 2173. doi:10.3390/app14052173.

[8] J. Lin, T. Lan, B. Zhang, et al., Multi-scenario cellular KPI prediction based on spatiotemporal graph neural network, IEEE Transactions on Automation Science and Engineering (2024). doi:10.1109/TASE.2024.3416912.

[9] J. Xiao, Y. Cong, W. Zhang, et al., A cellular traffic prediction method based on diffusion convolutional GRU and multi-head attention mechanism, Cluster Computing 28 (2025) 125. doi:10.1007/s10586-024-04848-y.

[10] S. Liu, M. He, Z. Wu, P. Lu, W. Gu, Spatial–temporal graph neural network traffic prediction based load balancing with reinforcement learning in cellular networks, Information Fusion 103 (2024) 102079. doi:10.1016/j.inffus.2023.102079.

[11] K. Yang, J. Yang, C. Shen, Average reward reinforcement learning for wireless radio resource management, in: 2024 58th Asilomar Conference on Signals, Systems, and Computers, IEEE, 2024, pp. 1188–1193.

[12] B. Adhikari, A. S. Khwaja, M. Jaseemuddin, A. Anpalagan, A. Nallanathan, Energy efficient RIS-assisted UAV networks using twin delayed DDPG technique, IEEE Transactions on Wireless Communications 23 (12) (2024) 18423–18439. doi:10.1109/TWC.2024.3468162.

[13] M. Wang, X. Liu, F. Wang, Y. Liu, T. Qiu, M. Jin, Spectrum-efficient user grouping and resource allocation based on deep reinforcement learning for mmWave massive MIMO-NOMA systems, Scientific Reports 14 (2024) 8884. doi:10.1038/s41598-024-59241-x.

[14] P. Pi, G. Ren, Soft actor–critic optimization for efficient NOMA uplink in intelligent vehicular networks, Physical Communication 68 (2024) 102581. doi:10.1016/j.phycom.2024.102581.

[15] K. Qi, Q. Wu, P. Fan, N. Cheng, W. Chen, J. Wang, K. B. Letaief, Deep-reinforcement-learning-based AoI-aware resource allocation for RIS-aided IoV networks, arXiv:2406.11245 (2024).

[16] S. M. Azimi-Abarghouyi, V. Fodor, Scalable hierarchical over-the-air federated learning, IEEE Transactions on Wireless Communications 23 (2024). doi:10.1109/TWC.2024.3350923.

[17] M. F. Pervej, R. Jin, H. Dai, Hierarchical federated learning in wireless networks: Pruning tackles bandwidth scarcity and system heterogeneity, IEEE Transactions on Wireless Communications 23 (9) (2024) 11417–11432. doi:10.1109/TWC.2024.3396873.

[18] J. Shao, F. Wu, J. Zhang, Selective knowledge sharing for privacy-preserving federated distillation without a good teacher, Nature Communications 15 (2024) 349. doi:10.1038/s41467-023-44383-9.

[19] Y. Liang, Q. Chen, G. Zhu, H. Jiang, Y. C. Eldar, S. Cui, Communication-and-energy efficient over-the-air federated learning, IEEE Transactions on Wireless Communications 24 (1) (2025) 767–782. doi:10.1109/TWC.2024.3501297.

[20] B. Wu, F. Fang, X. Wang, Joint age-based client selection and resource allocation for communication-efficient federated learning over NOMA networks, IEEE Transactions on Communications 72 (1) (2024) 179–192. doi:10.1109/TCOMM.2023.3326774.

[21] H. Li, J. Chen, W. Xu, M. Liu, Knowledge distillation for asynchronous

federated reinforcement learning in IoT, Knowledge-Based Systems 310 (2025) 113008. doi:10.1016/j.knosys.2025.113008.

[22] Y. Dai, L. Lyu, N. Cheng, M. Sheng, J. Liu, X. Wang, S. Cui, L. Cai, X. Shen, A survey of graph-based resource management in wireless networks — part II: Learning approaches, IEEE Transactions on Cognitive Communications and Networking 11 (4) (2024). doi:10.1109/TCCN.2024.3508777.

[23] Z. Liu, J. Zhang, E. Shi, Z. Liu, D. Niyato, B. Ai, X. Shen, Graph neural network meets multi-agent reinforcement learning: Fundamentals, applications, and future directions, IEEE Wireless Communications 31 (2024) 39–47. doi:10.1109/MWC.015.2300595.

[24] M. Ji, Q. Wu, P. Fan, N. Cheng, W. Chen, J. Wang, K. B. Letaief, Graph neural networks and deep reinforcement learning based resource allocation for V2X communications, IEEE Internet of Things Journal (2024). doi:10.1109/JIOT.2024.3468162.

[25] M. Marwani, G. Kaddoum, Graph neural networks approach for joint wireless power control and spectrum allocation, IEEE Transactions on Machine Learning in Communications and Networking 2 (2024) 717–732. doi:10.1109/TMLCN.2024.3408723.

[26] B. Li, L.-L. Yang, R. G. Maunder, S. Sun, P. Xiao, Heterogeneous graph neural network for power allocation in multicarrier-division duplex cell-free massive MIMO systems, IEEE Transactions on Wireless Communications 23 (2) (2024) 962–977. doi:10.1109/TWC.2023.3325973.

[27] G. Bianchi, Performance analysis of the IEEE 802.11 distributed coordination function, IEEE Journal on Selected Areas in Communications 18 (3) (2000) 535–547. doi:10.1109/49.840210.

[28] ns-3 Consortium, ns3-ai: Reinforcement learning interface for ns-3 (2023).
URL https://github.com/hust-diangroup/ns3-ai

[29] D. Kotz, T. Henderson, I. Abyzov, J. Yeo, Dartmouth campus WiFi traces 2018 (2018). doi:10.15783/C7F59T.